

Ossim: OS-Level Support for Fast End-to-End Compute Cluster Evaluation

Abstract

Datacenter research increasingly depends on end-to-end evaluation across CPUs, devices, networks, and emerging interconnects such as CXL. Physical testbeds are often impractical due to scale, cost, or hardware unavailability, while existing simulators expose sharp trade-offs: detailed models achieve fidelity but run prohibitively slowly, whereas emulators execute quickly but lack performance realism. Recent work on live simulation has shown that executing unmodified software on hardware with controlled timing can deliver both speed and accuracy, but current systems are limited to narrow domains such as GPU clusters and single-host hardware-software stack.

This project proposes OS-level support for fast, modular datacenter simulation by extending the Linux kernel, QEMU/KVM, and libvirt. We will introduce a simulation-aware scheduler and virtual clock to bound time skew, develop fine-grained delay-injection mechanisms for memory, I/O, and network modeling for QEMU/KVM, and augment libvirt with simulation orchestration primitives.

The resulting open-source framework will enable scalable, efficient execution of unmodified software stacks while providing performance insights. It will also provide the first simulation platform capable of running full distributed workloads with CXL-enabled disaggregated memory.

1 Motivation

Evaluating software-hardware co-design of modern datacenter systems requires comprehensive, end-to-end experimentation under realistic workloads. Yet full testbeds are rarely available: new hardware may not exist at research time, large deployments are costly, and rapid iteration is infeasible. Simulation offers an alternative, but current approaches exhibit fundamental limitations.

Frameworks such as SimBricks [8] demonstrate the value of modular simulation by interconnecting hosts, devices, and network simulators to support end-to-end evaluation. Such

a framework enables modularity and high fidelity while supporting execution of unmodified software stacks. However, in our target setting, it inevitably inherits the core shortcomings of its component simulators, which undermine practical utility.

Slow simulation speed. Detailed simulators such as gem5 [6] and Simics [7] provide cycle-level fidelity, which is essential to microarchitectural studies but may not be necessary for evaluating full-stack distributed applications. This level of detail imposes substantial computational overhead. For example, in the example OpenStack [11] cluster setup in Figure 1, simulating a 1-second single-threaded MySQL Sysbench requires more than two weeks. Such costs make large-scale deployment studies and iterative development workflows impractical.

Incomplete hardware functionality. Many simulators lack support for the latest hardware features due to engineering complexity or commercial restrictions. For instance, gem5 does not implement x86 virtualization extensions, forcing simulated cluster in Figure 1 to rely on alternative backends (e.g., TCG) instead of KVM for running VMs. This diverges significantly from real-world clusters that depend on hardware-assisted virtualization. First, latest hardware features are not always available in simulator implementations because of reasons such as extra engineering effort and commercial concerns. In addition, to keep simulation (relatively) tractable, simplified CPU models (e.g., SimpleTimingCPU) are often used, but these omit critical features such as pipelining and out-of-order execution. The result is unrealistic performance behavior that fails to align with modern hardware.

Thus, detailed simulation compromises both efficiency and accuracy in our target setting by modeling unnecessary details while omitting key system-level features.

Recent advances such as and Phantora [15] and NEX [10] highlight the potential of *live simulation*, where the above problems are resolved by executing unmodified software di-

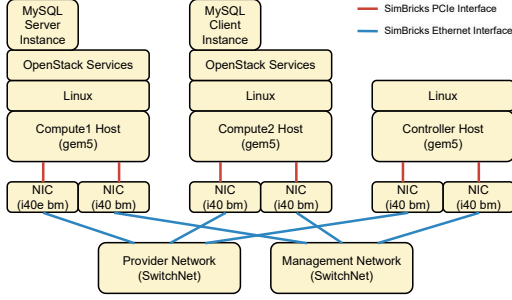


Figure 1: Simulating an OpenStack cluster with SimBricks by interconnecting gem5 instances, Intel I40e behavioral models and simple Ethernet switch simulators.

rectly on hardware (e.g., GPUs and CPUs), while coordinating timing virtually. However, current approaches remain limited in scope: they primarily target specialized domains such as GPU training clusters or single-host system stacks. There is a clear need for a general framework that achieves realism, speed, scalability and modularity simultaneously in datacenter contexts. Moreover, they largely emphasize computation, leaving the modeling of communication across components—essential for distributed systems—an open problem. Addressing this gap is especially critical for evaluating next-generation datacenters, such as those enabled by CXL [5] with tiered and disaggregated memory.

2 Approach

To overcome the limitations of detailed simulators and the narrow scope of existing live simulation systems, we propose a new framework for end-to-end datacenter evaluation that introduces OS-level support for timing, performance modeling, and orchestration. Our key insight is that by embedding simulation control into the operating system and virtualization stack, we can execute unmodified software stacks at near-native speed while maintaining accuracy and scalability.

At the kernel level, we design a simulation-aware scheduler and virtual clock abstraction. This scheduler bounds time skew across VMs and devices by combining preemptive and cooperative mechanisms. A global virtual clock similar to NEX [10] coordinates distributed components, while local clocks provide lightweight synchronization points for neighboring components. This design ensures approximately consistent progression of different components without incurring the prohibitive overhead of cycle-accurate simulation.

At the virtualization level, we extend QEMU/KVM [9, 14] with fine-grained mechanisms for performance modeling. Specifically, we employ OS-level primitives such as trapped page faults and I/O intercepts to introduce controlled delays that emulate memory hierarchies, I/O bandwidth limits, and network latency. Unlike detailed simulators, which model every microarchitectural feature, our approach focuses on enforcing performance constraints at system-relevant abstrac-

tion levels, enabling accurate evaluation of distributed applications with minimal overhead.

At the orchestration level, we extend libvirt [17] with simulation-specific capabilities. These extensions expose APIs for global clock control, declarative performance configurations, and distributed coordination of VMs and devices. This orchestration layer may also provide facilities for experiment lifecycle management, topology construction, and metrics collection, enabling reproducible and scalable simulation workflows.

Together, these components constitute a modular and extensible simulation framework. The system retains the flexibility of existing modular simulators, while addressing their fundamental limitations: it achieves speed by executing directly on hardware, accuracy through targeted performance modeling, and scalability through OS-level orchestration.

As a case study, we will apply this framework to simulate CXL-enabled datacenters with tiered and disaggregated memory to study the impact of CXL memory on distributed applications such as big data frameworks [1–4] and distributed databases [12, 13]. By combining native execution with controlled memory access latencies, we can emulate CXL semantics such as remote memory access and heterogeneous latency tiers, providing the first platform capable of running full software stacks on such architectures. This will enable exploration of new OS policies, runtime strategies, and distributed applications before CXL hardware is widely available.

In the longer term, the framework can be extended to support a broader range of hardware for live simulation by incorporating emerging accelerator virtualization techniques, such as XSched [16].

3 Contributions

This project will deliver:

- A kernel-integrated virtual clock and scheduler for consistent, live simulation.
- Performance modeling mechanisms in QEMU/KVM.
- Simulation-specific mechanisms and orchestration APIs for libvirt.

Together, these constitute an OS-level infrastructure for fast, modular datacenter simulation. By embedding the design within widely used open-source platforms, the framework will benefit a broad community. It will accelerate research on hardware-software co-design for datacenters, enable reproducible large-scale evaluation, and lower barriers to realistic experimentation. All contributions are planned to be released as open source, thereby enriching the QEMU, KVM, and libvirt ecosystems.

References

- [1] Apache Software Foundation. Apache Flink: Stream and Batch Data Processing Engine, 2025. Accessed: 2025-07-22.
- [2] Apache Software Foundation. Apache Hadoop: Open-Source Framework for Distributed Storage and Processing, 2025. Accessed: 2025-07-22.
- [3] Apache Software Foundation. Apache Hive: Data Warehouse Software for Reading, Writing, and Managing Large Datasets, 2025. Accessed: 2025-07-22.
- [4] Apache Software Foundation. Apache Spark: Unified Analytics Engine for Large-Scale Data Processing, 2025. Accessed: 2025-07-22.
- [5] CXL Consortium. Compute Express Link (CXL) Specification Revision 3.0. <https://www.computeexpresslink.org/download-the-specification>, 2022. Accessed: 2025-07-22.
- [6] gem5 Project. The gem5 simulator project. <https://www.gem5.org/>, 2025. Accessed: 2025-07-22.
- [7] Intel Simics. Simics full-system simulator. <https://www.intel.com/content/www/us/en/developer/articles/tool/simics-simulator.html>, 2025. Accessed: 2025-07-22.
- [8] Hejing Li, Jialin Li, and Antoine Kaufmann. Simbricks: end-to-end network system evaluation with modular simulation. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 380–396, 2022.
- [9] Linux Kernel Community. KVM: Kernel-based Virtual Machine. <https://www.linux-kvm.org/>, 2025. Accessed: 2025-07-22.
- [10] Jiacheng Ma, Jonas Kaufmann, Emilien Guandalino, Rishabh Iyer, Bourgeat Thomas, and George Candea. Fast end-to-end performance simulation of accelerated hardware–software stacks. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP ’25*, 2025.
- [11] OpenStack Foundation. OpenStack: Open Source Cloud Computing Infrastructure, 2025. Accessed: 2025-07-22.
- [12] Oracle Corporation. MySQL: Open-Source Relational Database Management System, 2025. Accessed: 2025-07-22.
- [13] PostgreSQL Global Development Group. PostgreSQL: The World’s Most Advanced Open Source Relational Database, 2025. Accessed: 2025-07-22.
- [14] QEMU Project. QEMU: The Fast Processor Emulator. <https://www.qemu.org/>, 2025. Accessed: 2025-07-22.
- [15] Jianxing Qin, Jingrong Chen, Xinhao Kong, Yongji Wu, Liang Luo, Zhaodong Wang, Ying Zhang, Tingjun Chen, Alvin R Lebeck, and Danyang Zhuo. Phantora: Live gpu cluster simulation for machine learning system performance estimation. *arXiv preprint arXiv:2505.01616*, 2025.
- [16] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. {XSched}: Preemptive scheduling for diverse {XPU}s}. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 671–692, 2025.
- [17] The libvirt Project. libvirt: The virtualization API, 2025. Accessed: 2025-07-22.